Manual Testing compact cheatsheet

STEP 1 Foundations

1.1 Software Testing Fundamentals

What software is and who builds it Software is a business need of a customer, turned into a working program. A bank such as ICICI wants its customers to log in, check a balance, transfer money and apply for a loan. The bank is not a software company, so it hands this need to one.

What testing means There are three common definitions, and a good answer in an interview mentions all three:

Manual testing Manual testing is the process of testing an application as per the client requirement, done by a test engineer by hand, without any tool running the steps.

Automation testing Automation testing is testing an application with the help of a tool. A tester writes or records a test script once, and the tool runs it again and again.

Programming and scripting languages

White box, black box and gray box Testing is grouped by how much of the inside the tester can see.

The seven principles of testing These seven principles (as taught by ISTQB) explain why testers work the way they do.

Why a late bug costs more A mistake in the requirement that is found during a requirement review costs a short conversation. The same mistake found after release means changing the design, the code, the tests and the documents, then shipping again.

Who does what in a software company A tester's career usually grows from test engineer to senior test engineer, test lead and test manager, or towards automation engineer and SDET (software development engineer in test).

QA, QC and testing: three different jobs People often use these words as if they mean the same thing. They do not, and interviewers love to ask about the difference.

Test oracles and the tester mindset To call something a bug, a tester must know what the right result is. Whatever tells you the right result is called a test oracle . The requirement is the main one, but it is never complete, so good testers use several:

The seven principles of testing

$10^{10} = 10{,}000{,}000{,}000$

EXAM Interviewers ask about black box testing more than anything else, because that is the daily work of a test engineer. The next pages go deep into white box testing and then into every level of black box testing.

1.2 SDLC and its Models

What SDLC is: build it like a house SDLC (Software Development Life Cycle) is the step by step procedure used to develop software. The notes compare it with building a house, because the steps line up almost exactly:

Service based and product based companies

The seven phases, one by one The HLD splits the application into modules . For Gmail: login, compose, inbox, sent items, outbox and logout. The LLD describes the objects inside each module. For the login page: the username box, the password box, the OK button and the Cancel button.

CRS, SRS and what makes a good requirement

The waterfall model The waterfall model is the basic model from which all the others grew. The phases run strictly one after another , and the output of one phase is the input of the next, like water falling down steps. It never flows back up.

The spiral model Waterfall's problem: an Excel like product with four modules (A: cells, B: operations, C: merging cells, D: macros), six months each, reaches the customer only after two years .

The prototype model The notes tell of a girl abroad who sees a photo of the boy before agreeing to meet him. The photo is a prototype : a sample that looks about 90% like the real thing.

The V and V model (V model) The V model pairs every document on the left with a level of testing on the right. While the left side is being written, testers review each document (verification) and prepare the test document for the matching level.

Agile and Scrum Agile is used in most companies today. The idea: build software quickly in small pieces, with the customer involved all the time . The notes show why.

Hybrid models and choosing a model A hybrid model combines two models to get the strengths of both. The usual pairs are spiral plus prototype (each module gets a prototype approved before it is built, module by module) and prototype plus V (a prototype is approved, then the V model's reviews ...

Kanban, scaled Agile and DevOps Scrum is not the only Agile way of working. Three more names appear in almost every job description today.

Where testing happens in each model Every model tests, but at different moments and in different amounts. Knowing this helps a tester plan, and it is a common interview question.

The spiral model

$T(A)$, then $T(B) + T(A)$, then $T(C) + T(A + B)$, then $T(D) + T(A + B + C)$

Agile and Scrum $250 \div 15 \approx 16.7$

TRAP Even after a careful hand over, problems appear in production. The two usual reasons are an INSTALLATION PROBLEM and CUSTOMER MISUSE . That is why maintenance is a phase of its own.

EXAM Review catches problems before any code exists. Reviewing a CRS that says "login, create mail, read mail, delete mail, view mail" shows that "read mail" and "view mail" mean the same thing, and that registration is missing even though login needs an account.

EXAM Common interview answers: team size 7 to 8, sprint length 10 to 15 working days, and a one year project (about 250 working days) has about

$$250 \div 15 \approx 16.7$$

so roughly 16 to 18 sprints. One weakness: a small team depends heavily on each member, so losing one person can hurt the project badly.

1.3 White Box Testing

What white box testing is White box testing (also called glass box, structural or unit testing) is testing the code itself, line by line, before the program is handed to the testers. It is done by developers, because they can see and understand the code.

Unit test programs A unit test program (UTP) is a small program that calls each function with chosen inputs and checks the result. When every UTP passes, the developer is confident the code is right and hands it over.

Path testing Path testing checks that every path through the code is executed at least once , so that each main function calls the right sub functions in the right order. Developers count the paths with respect to the code .

Condition testing Condition testing checks every condition for both its true and its false outcome. For a login: if the username and password are valid, show the home page, else show an error. One test with valid data and one with invalid data cover both outcomes.

Loop testing Loop testing checks that each loop runs exactly the number of times it should , and that it stops. The notes show three cases: a loop that runs fully, a loop left early with break , and a loop that must not run at all.

Statement and branch coverage Coverage measures how much of the code the tests actually ran:

Memory testing Memory testing checks that the code does not waste memory. The notes give an example: a game for an old Nokia phone had to fit in a tiny amount of memory, so every unused line had to go.

Response time testing Response time testing at code level finds the slow functions . A profiler such as Rational Quantify runs the code and shows which functions take the most time, drawn as thick lines in its call graph.

White box versus black box

Stronger coverage: MC/DC and mutation testing Statement and branch coverage show that code ran. Two stronger ideas check whether the tests would really notice a mistake.

What white box testing is $1 + 2$

Path testing $M = E - N + 2P$

Statement and branch coverage

$$\text{Statement coverage} = \frac{\textit{statements executed}}{\textit{total statements}} \times 100\%$$

TRAP Checking every function by hand is slow, inaccurate and tedious. A real program has thousands of lines and is changed every day. So developers write code to test their code.

EXAM Typical questions: What is white box testing? What are its types? What is a unit test program and who writes it? Why do we need both white box and black box testing? The short answer to the last one: developers prove the code does what they wrote, testers prove it does what the customer asked.

1.4 Functional, Integration and System Testing

Black box testing and the amount transfer example A black box tester never looks at the code. Everything starts from the requirement, and every object on the screen is tested against it. The notes use an amount transfer page:

Functional testing Functional testing (also called component testing) checks each component of a module, or each module, individually and independently.

Integration testing Integration testing checks the data flow, or interface, between two or more modules. After user A transfers 5,000 to user B, A's balance must drop by 5,000 and B's must grow by 5,000.

Types of integration testing, stubs and drivers When a module is missing during incremental testing, a stand in is used. A stub is a dummy called module (top down: the top module calls a lower module that is not built yet).

System testing System testing checks the end to end flow of the whole application : every module a real user would pass through, from logging in to logging out.

Smoke testing Imagine a new build arrives and the testers start deep testing module by module. On day five they reach compose and find it completely broken: a blocker .

Ad hoc and exploratory testing Real users never read the manual. Ad hoc testing (also called monkey testing) uses the application randomly , the way an unpredictable user would, usually at the end once the planned testing is done. The notes show what it finds on the amount transfer page:

User acceptance testing, alpha and beta UAT is testing done by the customer before accepting the product , to make sure it suits their real business. Buying shoes is the same: you check for damage, try them on, and walk a few steps before paying.

The levels of testing at a glance

Positive, negative and end to end testing Every level of testing mixes two attitudes:

Testing below the screen: APIs and system integration Modern applications are built from many services that talk through APIs (application programming interfaces). The screen is only the last step. A manual tester who can check the API layer finds bugs earlier and pinpoints them faster.

EXAM The notes add: do not volunteer the types of integration testing in an interview unless you are asked. Explain integration testing itself first.

1.5 Applications, Environments and Builds

The three parts of an application

Standalone, client server and web applications

Web application architecture A web application is split across three kinds of servers: web servers hold the pages, application servers run the business logic and database servers hold the tables. Each request only touches the servers it needs.

Development, test, staging and production environments An environment is a set of servers with the software and hardware needed to run the application. Each one has its own purpose and its own URL.

SQL for testers, and skipping a 30 day wait The overdraft is credited 30 days after approval. Testers cannot wait a month. The notes list the options:

The build process A build is the software converted from code into an installable application: a set of features and bug fixes installed on the test server to be tested for stability.

Respin, patch and blocker A blocker stops testing completely. There are two ways to unblock the testers:

Release, version and the road to production A release is the final, stable product handed over to the customer: the whole set of features that was developed, tested and found stable. It also names everything done from requirement to hand over.

Test data: realistic, safe and ready A test is only as good as its data. A transfer test needs accounts with the right balance, the right status and the right history. Test data management is the work of creating, protecting and refreshing that data.

Cloud, containers and environments on demand Not long ago a team shared one QA server for months. Today environments are often created in the cloud in minutes and thrown away after use.

1.6 Static Testing and Reviews

What static testing is Static testing means checking a work product without running it. Nobody clicks a button or enters a value. People read the document, the design or the code, and tools scan it, looking for mistakes.

What can be reviewed, and what reviews find Almost anything people write can be reviewed. If a person can read it and understand it, a person can check it.

Verification and validation These two words sound alike and appear in every interview. The classic way to remember them comes from Barry Boehm:

Reviewing a requirement like a tester A tester is often the first person who asks "how would I prove this works?" That question exposes weak requirements. A good requirement has these qualities:

The review process in five steps The ISTQB Foundation syllabus (version 4.0) describes a review as five activities. Small informal reviews squeeze them into a few minutes. A formal inspection follows every step and writes everything down.

Who does what in a review One person can hold several roles in a small team. In a formal inspection each role is a different person. Tap a role in the diagram to read its job.

Four types of review, from a chat to an inspection Reviews range from a quick look by a colleague to a strict, measured inspection. The more formal the review, the more it costs and the more defects it finds per page.

Static analysis: tools that read for you Static analysis is static testing done by a tool. The tool reads the code or a document and reports problems, without running anything. Developers use it most, but testers read its reports and use similar tools on their own documents.

Making reviews work in a real team Reviews fail when they feel like an exam for the author or a waste of time for everyone else. These habits keep them useful:

EXAM In the V model, the left side (reviewing CRS, SRS, HLD and LLD) is mostly verification, and the right side (running the tests, ending with UAT) adds validation.

TRAP Criticise the work, never the person. "Requirement 4.2 has no maximum length" helps. "You forgot the maximum length again" makes the author defend the document instead of fixing it.

EXAM Who leads a walkthrough? The author. Who must never lead an inspection? The author. This one pair of facts answers many exam questions.

STEP 2 Test design and defects

2.1 Test Case Design and Documents

Why test documents, and how they are made Test documents are written by testers before test execution, while developers are writing the business logic , so that they are ready when the first build arrives.

Test scenarios A test scenario is a high level, one or two line statement of something to test. Each one has a do part and a check part, and scenarios are written module by module. They help a tester understand the flow without worrying yet about exact values.

The test case template A test case is a detailed document with the step by step procedure, the inputs and the expected result for every scenario. Each company has its own template, usually an Excel sheet supplied by the test lead. It has three parts:

Error guessing, equivalence partitioning and boundary values The hardest part of writing test cases is maximum test coverage with a small number of values. Three techniques from the notes:

Decision tables, state transitions and pairwise testing Three more techniques complete a tester's toolbox:

Functional, integration and system test cases The same template holds all three levels. Only the scope changes:

Test case review After every tester finishes writing, test cases are reviewed by another tester, for two reasons: accuracy (maximum coverage) and proper flow , so anyone can reuse them.

The repository and the RTM Approved test cases go into the test case repository , a central place created by the test lead. Once stored they are baselined : written, reviewed and approved, and not changed casually. Smoke test cases are not written separately.

The test execution report The test execution report (TER) is prepared by the lead and handed to the customer. It shows how stable the product is: test cases written, executed, passed and failed per module, with percentages.

STLC: the software testing life cycle

Use case testing and checklist based testing A use case describes how an actor (a user or another system) reaches a goal, step by step. It has a main flow where everything goes well, alternative flows where the user takes another valid path, and exception flows where something goes wrong.

Writing test cases that others can run A test case is written once and run many times, often by someone else. These habits make it useful for years.

Error guessing, equivalence partitioning and boundary values

$a - 1, a, a + 1, b - 1, b, b + 1$

EXAM Interview tip from the notes: if asked to write test cases for an unknown application (ERP, insurance), ask for the requirement first. For a known one (Gmail), write scenarios. For an object (a pen, a chair, an ATM), think of functional, integration, system, compatibility, performance and usability checks.

TRAP DRAWBACK OF THE RTM: it only ensures that every requirement has at least one test case. It does not guarantee that all the test cases a requirement needs are there.

EXAM Typical interview numbers from the notes: about 3 to 4 test cases written a day and 2 to 3 executed, depending on complexity, and roughly 90 to 120 test cases for a two to three month project.

2.2 Defect Management

Defect, bug, error, issue and failure A chain links them: a developer makes an error , the code now contains a defect , and when that code runs it can cause a failure . Defects come from three kinds of implementation:

The defect life cycle Every bug moves through a fixed set of statuses. Only certain moves are allowed, and each move is made by a particular person.

Who gets the bug

Invalid and duplicate The developer does not accept the bug because it is not against the requirement. There are two outcomes:

Not reproducible The developer accepts that it might be a bug but cannot make it happen. The notes list six reasons. Pick one in the widget to compare the tester's setup with the developer's.

Can't fix, deferred and RFE

Severity and priority The developer works through the queue by priority first and severity second:

Writing the bug report A good bug report lets a developer reproduce the bug without asking a single question. The template fields:

Bug tracking tools and queries The tool generates the bug id, fills in the reporter and date, and offers drop downs for module, status, severity and priority.

The defect triage meeting When many bugs arrive at once, not all can be fixed at once. Triage , a word borrowed from hospital emergency rooms, means deciding quickly which bugs need attention first. Most teams hold a short triage meeting daily or a few times a week during a test cycle.

Root cause analysis: 5 whys and the fishbone Fixing a bug removes one defect. Finding its root cause stops the same kind of bug from coming back. Two simple techniques are used everywhere, from car factories to software teams.

EXAM Classic interview pair: HIGH SEVERITY, LOW PRIORITY (a crash in a report used once a year) and LOW SEVERITY, HIGH PRIORITY (the company name misspelled on the home page). Severity and priority are related but not the same.

TRAP Keep the description factual: what you did, what you saw, what you expected. Never write "the developer broke the inbox again".

STEP 3

Regression and non functional testing

3.1 Regression, Retesting and Automation

What regression testing is Each new release adds features, removes some, or modifies others. All three mean the code changes, and any code change can break something that already worked.

Unit, regional and full regression Follow the dependencies one step at a time to find the impact area of a change:

Impact analysis and the regression suite No single person sees every impact, so three groups each send a report:

Retesting versus regression Regression happens at two levels. Across releases because of new or changed features, and across builds because of bug fixes: B001 has bugs, the developer changes the code to fix them, and on B002 the tester checks the fixes and the areas around them.

Why regression moves to automation Regression is the most repetitive work in testing, which makes it the best candidate for automation. A tool runs the same steps in seconds, does not get bored, and can run every night.

The automation life cycle A night of script runs on a new build, and what the team does with each failure:

What cannot be automated

The classic comparisons

Choosing what to automate first Nobody automates everything at once. A simple score helps a team pick the cases that pay back fastest:

Flaky tests and why they matter A flaky test passes and fails on the same code without any change. Run it ten times and you might see eight green and two red. Flaky tests are dangerous because people learn to ignore red results, and then a real failure is ignored too.

EXAM If an interviewer asks which regression you do, the usual answer is REGIONAL REGRESSION , with the impact area found by impact analysis.

TRAP Automation does not replace testers. Someone still writes the test cases, reviews failures, and does the exploratory and usability testing that needs human judgement.

3.2 Non-functional Testing

What non-functional testing covers

Functional testing asks whether each feature does what the requirement says. Non-functional testing asks how well the product does it: on which platforms, how easily, and how fast under real load.

Compatibility testing Compatibility testing checks the application on different software and hardware platforms, because real users do not all have the same machine.

Virtual machines and the compatibility process The notes compare three ways to get many platforms. Giving each tester one OS means nobody can check another. Installing several OS on each machine means restarting all day.

Usability testing Usability is user friendliness: important features are easy to reach, the language is easy to understand, the look and feel is consistent, navigation takes few steps, and the effort to learn is small.

Performance testing Performance testing checks how the application behaves when load is applied. Three factors matter:

Bottlenecks and tuning If the average response time is 6.3 seconds against a target of 5, there is a bottleneck . Performance bugs are not reproduced like functional bugs. They are analysed and tuned:

Load, stress, scalability, volume and soak At 1,000 users the system is now 2.5 seconds slower than before the stress. It did not recover, so the team looks for a memory leak.

The quality model: ISO/IEC 25010:2023 "Non functional" is a big bag. The international standard ISO/IEC 25010 sorts product quality into named characteristics, and its 2023 edition updated the list. Teams use it as a checklist so that no kind of quality is forgotten in the test plan.

Reliability, recovery and installation testing Users forgive a slow day. They do not forgive lost money or lost data. These tests check that the product keeps working, and comes back safely when something breaks.

Performance testing

$1000 \times 30 \times 24 = 720{,}000$

Load, stress, scalability, volume and soak

$\text{users} = \text{throughput} \times (\text{response time} + \text{think time})$

Reliability, recovery and installation testing

$$\text{availability} = \frac{MTBF}{MTBF + MTTR}$$

EXAM Little's law links the numbers:

$$\text{users} = \text{throughput} \times (\text{response time} + \text{think time})$$

. With 1,000 users, a 5 second response and 15 seconds of think time, the server must handle

$$\frac{1000}{20} = 50$$

requests per second.

3.3 Security, Accessibility, Localization and Mobile Testing

Security testing: what it protects Security testing checks that the application protects its data and its users from people who want to misuse it. A functional tester asks "does it work for the right user?" A security tester also asks "what happens when the wrong user tries?"

The OWASP Top 10 (2025) for testers OWASP, the Open Worldwide Application Security Project, publishes a list of the ten most serious risks for web applications, based on data from thousands of real applications. The newest edition is the OWASP Top 10:2025 .

Accessibility: software for everyone Accessibility testing checks that people with disabilities can use the product. That includes people who are blind or have low vision, are deaf, cannot use a mouse, have tremors, dyslexia or memory problems.

How to test accessibility by hand Automated tools such as axe, Lighthouse and WAVE are a good first step, but they catch only part of the problems. They can see that an image has no alt text. They cannot tell that the alt text "image123" is useless. A manual pass is always needed.

Globalization: one product, many countries When a product is sold in many countries, two jobs appear:

Mobile app testing A phone is not a small computer. It moves between networks, runs on a battery, gets calls in the middle of a task, and comes in thousands of models. Mobile testing adds these questions to normal functional testing.

Privacy and data protection testing Laws now give people rights over their personal data, and products must prove they respect them. Testers check these rights like any other requirement.

How to test accessibility by hand

$$\text{contrast ratio} = \frac{L_{light} + 0.05}{L_{dark} + 0.05}$$

TRAP Only test security on systems you have written permission to test, and only on test environments. Trying these checks on a real website you do not own can be a crime, even with good intentions.

STEP 4 Planning and closure

4.1 The Test Plan

What a test plan is A test plan is a document written by test leads and managers that describes every testing activity the testers will carry out.

Objective, scope and approach Writing down what is out of scope matters as much as what is in. It stops arguments later about why something was not tested.

Schedule and methodology 4. Schedule gives each activity a duration and a date: reading the requirement, writing scenarios, writing test cases, review, execution cycles and release. 5.

Roles and responsibilities

Test environment and deliverables

Assumptions, risks and mitigation Sections 9 to 11 belong together: what might go wrong, what it would cost, and what the team will do about it.

Tools

Entry and exit criteria Entry criteria are the conditions to start testing: coding complete, the build installed on the test server, test documents ready, and people and tools in place.

Parallel testing and the whole journey Parallel testing feeds the same inputs to the old and the new version and reports every difference. It checks that the new version is correct, that it behaves like the old one where it should, that data integrity holds, and whether data formats changed.

Test strategy and test approaches A test strategy is the general way an organisation or a programme tests: which levels, which kinds of testing, which tools, and how risk is handled. A test plan applies that strategy to one project or release, with dates, names and scope.

Risk based testing in practice Time is always short. Risk based testing spends it where failure would hurt most. Each risk gets a likelihood (how probable a defect is) and an impact (how bad a failure would be), usually from 1 to 5. Their product is the risk level .

Schedule and methodology

$$\frac{93}{4} \approx 24$$

Risk based testing in practice

$$\text{risk level} = \text{likelihood} \times \text{impact}$$

EXAM From the notes: the test plan sets 96% as the release pass rate, the TER shows 98%, so the product can be released.

4.2 Test Estimation, Metrics and Reporting

Why testers measure A number is useful only if it helps someone decide something. "We ran 4,000 test cases" sounds busy, but it does not say whether the product is safe to release. "All 38 high risk requirements pass, and 2 major bugs are open in payments" does.

Estimating test effort An estimate is a careful guess with its reasons written down. No technique is perfect, so experienced leads use two and compare.

Measuring test progress During execution, the lead checks progress every day. A few numbers tell most of the story:

Defect metrics that show how good testing is These numbers measure the testing process itself. They are asked about in interviews and in every release review.

Coverage: what have we really checked "How much have we tested?" has several honest answers, and each one sees something different.

Test status and completion reports The ISO/IEC/IEEE 29119 standard names two main reports. A test status report (also called a progress report) goes out regularly, often daily or weekly. A test completion report (often called the test summary report) closes a level or a release.

Test closure: finishing properly When a release ships, testing is not simply dropped. Test completion (closure) activities make the next release easier:

Estimating test effort

$$\text{E} = \frac{O + 4M + P}{6} \quad \text{and} \quad \text{SD} = \frac{P - O}{6}$$

TRAP A metric used to judge people stops measuring the work. If testers are rewarded per bug, the team gets many tiny duplicate bugs. If developers are punished per bug, bugs get argued away. This is Goodhart's law: when a measure becomes a target, it stops being a good measure.

EXAM DRE is calculated with valid defects only. Rejected and duplicate bugs are not real defects, so remove them before you divide.

STEP 5 Modern testing practice

5.1 Agile, DevOps and Shift Left Testing

The whole team owns quality In Agile, quality is not one department's job. The whole team , developers, testers, the product owner and designers, is responsible for it. The tester is still the expert in finding risks, but now shares that skill with everyone, all the time.

User stories and acceptance criteria A user story is a small piece of value, written from the user's point of view:

Definition of Ready and Definition of Done Two short checklists stop half baked work from entering or leaving a sprint.

The agile testing quadrants Brian Marick's agile testing quadrants , made popular by Lisa Crispin and Janet Gregory, sort every kind of test by two questions: does it face the business or the technology , and does it guide the team while building or critique the product after it is ...

The test pyramid Mike Cohn's test pyramid says how many checks to put at each level. Many small fast checks at the bottom, fewer slow checks through the screen at the top.

CI, CD and quality gates Continuous integration (CI) means every change a developer pushes is built and checked automatically, many times a day. Continuous delivery (CD) means the product is always in a state that could be released with one click.

Shift left and shift right Shift left means moving testing earlier: reviewing stories, writing examples before code, running checks on every commit. Shift right means continuing to learn after release, from real users in production. Modern teams do both.

Exploratory testing that the team can trust In fast moving teams, scripted test cases cannot keep up with every change. Exploratory testing fills the gap: the tester learns, designs and runs tests at the same time.

TRAP "Done but not tested" does not exist. If testing is not finished, the story is not done, and it does not count in the sprint velocity.

5.2 AI in Testing and Testing AI

What generative AI is, in plain words A large language model (LLM) has read a huge amount of text and learned which words usually follow which. When you ask it something, it writes the most likely next words, one after another.

AI as a testing assistant Used well, AI removes the slow typing from testing and leaves the thinking to you. These are the tasks where testers gain the most today:

Writing prompts that give useful answers A vague prompt gets a vague answer. A good prompt gives the model the same information you would give a new tester on their first day.

The risks of using AI in testing The ISTQB syllabus on testing with generative AI (CT-GenAI, released in July 2025) lists the main risks testers must manage. In everyday words:

AI inside testing tools Many test tools now advertise AI features. Know what each one really does, so you can judge the claims.

Testing a product that contains AI Classic testing compares the actual result with one expected result. An AI chatbot, a recommendation engine or a fraud model does not give one fixed answer.

Quality risks special to AI Beyond "is the answer right?", AI systems need checks that normal software rarely needs:

Growing as a tester in the AI era AI changes how testers spend their time, not whether testing is needed. Someone still has to decide what matters, what could go wrong and whether the evidence is good enough. These skills grow in value:

TRAP Never paste customer data, passwords, internal code or confidential documents into a public AI tool. Use only the tools and settings your company has approved.